

The Theory in Practice

*What Building a Core Banking System with Claude Code
Actually Taught Me About AI Mapping*

A month ago I published a piece arguing that mapping human activities onto AI models is not an exercise in imitation but in clarification — that before you can safely externalise work to an AI agent, you must first model your own activities coherently, decompose roles into functions, and treat the artefacts you give an AI as semantic contracts rather than narrative conveniences. Since then I have been testing that theory against something concrete: building a core banking system, from scratch, using Claude Code as a development partner on a personal project I am calling Nucleus.

This is what I found.

The domain

Core banking is a good stress test for AI-assisted development. The domain is old, well-understood in broad strokes, and yet semantically precise in ways that punish vagueness. A balance is not a number — it is a deterministic calculation: the result of applying a set of ledger entries up to time T1, as observed from time T2. An interest rate is not a property — it is an effective-dated value that governs resolution only within an open processing period. A payment is not a transaction — it is an ISO 20022 message exchanged with an external counterparty under scheme rules that are not negotiable.

If you build these things correctly, they are robust to decades of change. If you get the semantics wrong, the errors accumulate silently and surface as financial discrepancies that are expensive and sometimes impossible to unwind.

The domain also involves multiple stakeholders with incompatible perspectives: the savings team that configures products, the reconciliation team that must balance the ledger to the penny, the operations team that monitors processing jobs, the external payment rail that speaks only ISO 20022. Each of these stakeholders has distinct goals, distinct constraints, and distinct integration patterns. None of them is reducible to any of the others.

This is, in other words, precisely the kind of domain where semantic drift is costly, where fluent-but-ungrounded output from an AI agent would be actively dangerous, and where the mapping problem is real.

The first thing the framework had to do was force me to model myself

My original piece argued that humans must model their own activities before they can map them onto AI agents, and that in most organisations this modelling has never been made explicit. Roles are understood through convention, apprenticeship, and social feedback. Outputs are evaluated pragmatically rather than against declared semantic contracts.

The first thing I discovered in practice is that this is true of individuals as well as organisations, and that the discipline of writing it down is more valuable than you expect.

Before writing a line of code — before even describing the technical stack to Claude Code — I wrote persona documents. Not user personas in the marketing sense, but detailed characterisations of every stakeholder in the system: their goals, their constraints, their integration patterns, and crucially their interests across each domain area. Eleven personas in total, ranging from Sasha from Savings (an automated configurer value stream) to Casey the Customer (the indirect beneficiary who never touches the system directly) to Parker from Payments (the external Faster Payments rail provider who speaks only ISO 20022 and is indifferent to Nucleus's internal structure).

Writing those documents forced me to answer questions I had been carrying as tacit knowledge. What exactly does the reconciliation team need from a ledger entry event? Not "the details" — the specific fields: debit/credit indicator, amount, currency, value date, effective date, entry type, account identifier, transaction identifier, and the identifier of the entry being corrected if this is a reversal. Until that is written down, it is not a semantic contract. It is a wish.

What does the risk function need when it places a financial crime restriction on an account? A restriction event, yes — but one that may be subject to tipping-off rules under the Proceeds of Crime Act, which means the event model needs a structured confidentiality classification, and the Kafka topic access controls must enforce read permissions by restriction type. Until that is written down, it is an assumption waiting to become a compliance gap.

The persona documents did not teach Claude Code about the domain. They taught me that I had not yet finished thinking about it.

Mapping as decomposition: the role files

My original piece argued that effective mapping proceeds from activities to functions, not from roles to agents — that the correct unit of mapping is not "what a person is" but "what transformations they perform." In practice, this is what the role file system implements.

I did not create an "AI developer" role. I created three distinct functional modes: architect, story author, and TDD implementor. Each has a defined entry condition, a defined output type, and explicit constraints on what it may and may not produce.

The architect role produces domain models and ADR candidates. It may not write production code, tests, or implementation proposals. It does not implement; it reasons about structure.

The story author role produces user stories with Gherkin acceptance criteria. It may not write production code or implementation proposals. It does not implement; it translates domain understanding into verifiable requirements.

The TDD implementor role produces production code and tests, in strict red-green-refactor order. It may not author stories or make architectural decisions. If an architectural question surfaces during a TDD session, it is parked and addressed in a subsequent architecture session.

The prohibition on mixing roles within a single session is the enforcement mechanism. A session that starts as story authoring and drifts into architecture produces lower quality output in both modes — not because Claude Code is

incapable of doing both, but because the authority boundaries become undefined, and undefined authority boundaries are precisely what my original piece identified as the source of authority leakage.

In practice, this discipline is harder to maintain than it sounds. There is a constant temptation to resolve an architectural question inline during a TDD session, because the answer seems obvious and the test is waiting. The answer may indeed be obvious, but it will not be documented, and it will become an implicit assumption embedded in production code that no one wrote down. The discipline of stopping, parking the question, and returning to it in an architecture session is not bureaucratic overhead. It is the mechanism by which the semantic contract stays intact.

Artefacts as semantic contracts: the CLAUDE.md hierarchy

My original piece argued that artefacts suitable for AI context must make distinctions explicit rather than implicit, encode intent and constraint rather than outcome alone, and be consumable without inference or background knowledge. The CLAUDE.md hierarchy is an architecture for doing exactly this.

Claude Code loads a hierarchy of markdown files at session start. At the top, a global file captures invariant engineering identity: the TDD cycle is non-negotiable, BDD acceptance criteria are written in Gherkin, domain modelling precedes implementation. Below that, a project-level file captures technical orientation: the stack, the build commands, the codebase conventions. Below that, role files and reference documents are loaded on demand via explicit file references.

The key architectural insight is that the context window is not a limitation to work around — it is a constraint that forces you to be explicit about what any given session needs to know. A session that loads everything produces worse results than a session that loads exactly what is needed. This is not because Claude Code cannot handle a large context — it is because loading irrelevant material dilutes the signal of the relevant material, and diluted signal produces outputs that are locally plausible but globally misaligned.

This is the "fluent but ungrounded" failure mode from my original piece, instantiated in a specific mechanism. The fix is not a better model. It is better context engineering.

The practical consequence is that every session requires a deliberate decision about what to load. An architecture session on the parameter value hierarchy loads the architect role file, the relevant persona documents (the configurer personas — Sasha, Liam, Maya — whose concerns the hierarchy exists to serve), and any prior architecture documents that establish settled ground. An account creation story session loads the story author role, the account-relevant personas, and the parameter hierarchy architecture document. A TDD session loads the implementor role, the story being implemented, and the architecture documents that define the domain model the implementation must express.

The discipline of selecting what to load is itself a form of self-modelling. If you cannot say what a session needs to know, you have not yet finished defining what the session is for.

What the theory predicted correctly

The failure modes are precisely those the theory anticipates. Authority leakage manifests as role drift: a TDD session that starts debating acceptance criteria, a story authoring session that quietly begins designing the implementation. Context accumulation producing lower quality output is the fluent-but-ungrounded problem instantiated in a specific mechanism. The human contributor guide I wrote for the project has a section called "Common Failure Modes" — re-reading it against the original theory, it is essentially an empirical catalogue of what happens when the mapping principles are violated.

Prior self-modelling is necessary, and its absence is expensive. The processing boundary architecture session — the session that established when a backdated configuration change is and is not permitted — took longer than any other session in the project because I had not yet thought through the full implications of effective-dated parameter values. The session did not produce the answer. It produced the questions I needed to answer. That is valuable, but it would have been more valuable earlier, and the only reason it was not earlier is that I had not yet modelled that aspect of my own domain understanding.

Fluent output and grounded output are different things, and the difference is not always visible in the output itself. Claude Code will produce plausible-sounding domain models, plausible-sounding acceptance criteria, and plausible-sounding production code in response to underspecified inputs. The plausibility is not a sign that the output is correct. It is a sign that the model is good at pattern-matching within the supplied context. Grounded output requires that the context itself be grounded — which is the responsibility of the human, not the model.

What practice adds to the theory

The session boundary as authority reset. The `/clear` command between stories is not just memory management. It is a deliberate reset of the authority scope. A session that has implemented three stories accumulates implicit assumptions about those stories that are not relevant to the fourth. Clearing enforces the principle that each functional unit — each story, each architecture session, each task — has a defined and bounded scope of authority. The session boundary is the architectural mechanism by which authority leakage is prevented from accumulating across units of work.

The context window constraint as a feature. Traditional software documentation fails not because it is too long but because it is too undifferentiated — everything is equally important and equally present, so nothing is. The forced scarcity of the context window requires you to make decisions about relevance that you would otherwise defer. These decisions are not overhead. They are the modelling work that the theory says must be done.

The role file as a repeatable semantic contract. A human working in a given role carries tacit knowledge about the scope and limits of that role that they acquired through experience and that they exercise through judgement. An AI agent has no such tacit knowledge and no such judgement. The role file is the mechanism by which the scope and limits of a functional mode are made explicit and repeatable. The same architect role file governs every architecture session, regardless of which domain area is being modelled. The constraints are identical. The outputs are comparable. The accumulation of role files across a project is the accumulation of explicit, reusable semantic contracts — something that human role definitions have almost never been.

What remains open

I want to be clear about the limits of what this experiment demonstrates.

This is a personal project, conducted by a single engineer with a specific methodological background, outside of any organisational context. The framework I have described was designed by someone who already understood what it needed to express. The discipline of role separation, session boundary management, and context selection was maintained by someone who had written the theory behind it. These are not transferable conditions.

The next question — the one this experiment has not answered — is whether the framework scales to an organisation. Several specific problems arise:

The global CLAUDE.md is personal. The engineering identity file at `~/claude/CLAUDE.md` captures one person's invariant practices: their TDD discipline, their BDD conventions, their domain modelling philosophy. An organisation has no equivalent single identity. Different engineers have different practices, different levels of domain understanding, and different relationships to the constraints the framework enforces. A framework that depends on a single, disciplined author does not generalise to a team without a mechanism for establishing shared conventions and enforcing them.

The persona and architecture documents must be maintained as the domain evolves. In a personal project, the author of the documents and the consumer of the documents are the same person. Inconsistencies are noticed quickly because they surface as friction in sessions that the author also runs. In a team, the author of a persona document and the engineer using it in a TDD

session may be different people, working at different times, with different understandings of what the document is supposed to convey. Document drift is a real risk, and the framework has no current mechanism for detecting or correcting it.

The discipline of role separation requires active maintenance. It is not a natural workflow. The temptation to resolve architectural questions inline, to absorb scope creep into the current story, and to treat the TDD session as the appropriate place to make design decisions does not go away. In a personal project, the human enforcing the discipline and the human experiencing the temptation are the same person. In a team, enforcement requires shared understanding, shared commitment, and — eventually — shared tooling.

None of these problems are insurmountable. But they are real, and pretending otherwise would undermine the honest utility of the experiment.

Conclusion

The theory holds. Mapping human activities onto AI models is indeed an exercise in clarification rather than imitation. The persona documents, the role files, the CLAUDE.md hierarchy, and the session discipline are all expressions of the same fundamental principle: that AI agents do not reduce the need for conceptual clarity, but expose its absence and scale its consequences.

What practice adds is specificity. The context window is not an obstacle but a forcing function. The session boundary is not a housekeeping convention but an authority boundary. The role file is not a prompt template but a repeatable

semantic contract. The discipline of loading only what a session needs to know is not parsimony but signal hygiene.

And the failure modes are exactly what the theory predicts. Role drift is authority leakage. Context accumulation is signal dilution. Plausible-sounding output from an underspecified context is fluency without grounding. Naming these precisely is the first step toward designing against them.

The experiment is ongoing. The core banking system is incomplete. The framework will encounter problems I have not yet imagined. But the theory has survived contact with a real domain, a real codebase, and a real development workflow — and in surviving, it has acquired some useful specificity.

That seems worth reporting.